ITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 2 — MICROSERVICES EDGE CASE TESTING

Gayuma, Jeremiah; Mamar, Kharl Benedict; Alacapa, Jesryl John; Barbas, Christian; Orbigoso, Mikhoel Runo
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise evaluated a microservice-based Student Information System composed of a Student Service (port 8001), Course Service (port 8002), and Enrollment Service (port 8003). I designed and executed my own curl-based test suite that covered normal CRUD operations and critical edge cases, including invalid input, missing resources, duplicate data, and dependency failures. All requests were issued with curl -i to capture both headers and JSON bodies, allowing me to verify HTTP status codes 200, 201, 400, 404, 409, 503, and 504 together with consistent ERROR_CODE fields. The results show that all three services respond correctly to client-side errors, and that only the Enrollment Service returns 503 and 504 because it is the only service that depends on the other services over HTTP. Overall, the system behaved in a predictable and robust way when subjected to the edge cases in my test plan.

I. INTRODUCTION

In real-world systems, most bugs and outages do not come from ideal "happy path" scenarios; they come from edge cases and failure conditions. Clients send malformed JSON, reference IDs that do not exist, or call an API while another service is offline or overloaded. A well-designed microservice must treat these situations as first-class cases, returning accurate HTTP status codes and structured error responses that clients can interpret programmatically.

The HTTP status code family provides a standard way to describe the outcome of each request. Successful operations use 2xx codes such as 200 OK for reads and updates or 201 Created for newly inserted resources. Client-side mistakes are signaled by 4xx codes: 400 Bad Request when the JSON body is missing required fields or contains invalid formats, 404 Not Found when a resource cannot be found, and 409 Conflict when a request would violate a uniqueness rule. Server-side problems are mapped to 5xx codes, most notably 503 Service Unavailable when a dependency cannot be reached and 504 Gateway Timeout when a dependency is too slow to respond.

Microservices introduce failure modes that monolithic systems rarely encounter. In a monolith, all logic and data access run in a single process, so there is no external HTTP dependency between components. In a microservice architecture, one service may need to call another service over the network, which introduces new ways to fail: the remote service might be offline, might crash during a request, or might respond after a long delay. In this laboratory, I focused on how my Student, Course, and Enrollment microservices behave

under these conditions, using curl to trigger edge cases and confirm that the services respond with the correct codes and error bodies.

II. METHODOLOGY / IMPLEMENTATION

A. System Architecture

The system is implemented as three independent Laravel-based microservices:

Student Service (port 8001) — Manages student records, including creating, reading, updating, and deleting students. It enforces constraints such as unique email addresses and basic validation of required fields.

Course Service (port 8002) — Manages courses, including codes, names, and other metadata. It enforces unique course codes and ensures that required fields are present.

Enrollment Service (port 8003) — Connects students and courses by creating enrollments and may also manage related data such as status, grades, or attendance. It uses HTTP calls to the Student and Course services to validate foreign IDs before inserting records.

Each microservice has its own database and routes, and they communicate over HTTP using JSON. The Enrollment Service is the only one that actively calls other services; the Student and Course services interact only with their own databases.

B. Environment Setup

Before running tests, each service is started on its own port. The typical setup is:

Start the Student Service on http://localhost:8001.

Start the Course Service on http://localhost:8002.

Start the Enrollment Service on http://localhost:8003.

I also ensure that the databases are in a known state before testing. On my setup, that means running the appropriate migration and seeding commands (for example, Laravel's migrate:fresh --seed for each service) so that a small set of initial students and courses always exist and the enrollment table is empty at the start of the run. This makes resource IDs predictable and avoids false test failures caused by leftover data from earlier experiments.

C. Test Design

Health and basic connectivity

curl.exe -i http://localhost:8001/api/health

```
PS C:\laragon\www\ITPFL\lab-2> curl.exe -i http://localhost:8001/api/health
HTTP/1.1 200 OK
Host: localhost:8001
Date: Tue, 24 Mar 2026 18:47:39 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"service":"Student Service","status":"running","port":8001,"database":"student_service.sqlite"}
```

curl.exe -i http://localhost:8002/api/health

```
PS C:\laragon\www\ITPFL\lab-2> curl.exe -i http://localhost:8002/api/health
HTTP/1.1 200 OK
Host: localhost:8002
Date: Tue, 24 Mar 2026 18:49:43 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"service":"Course Service","status":"running","port":8002,"database":"course_service.sqlite"}
```

curl.exe -i http://localhost:8003/api/health

```
PS C:\laragon\www\ITPFL\lab-2> curl.exe -i http://localhost:8003/api/health
HTTP/1.1 200 OK
Host: localhost:8003
Date: Tue, 24 Mar 2026 18:53:24 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"service":"Enrollment Service","status":"running","port":8003,"database":"enrollment_service.sqlite","depends_on":{"Student Service
":"http:\/\/localhost:8001","Course Service":"http:\/\/localhost:8002"}}
```

I begin with GET /api/health on each service to confirm that the processes are running and reachable. All three should return 200 OK with a simple JSON body.

Happy-path CRUD operations

I then perform straightforward create-and-read tests:

POST /api/students followed by GET /api/students/{id} to create and fetch a student.

curl.exe -i -X POST http://localhost:8001/api/students ^
  -H "Content-Type: application/json" ^
  -d "{\"name\":\"Juan Dela Cruz\",\"email\":\"juan@example.com\",\"phone\":\"09171234567\",\"year_level\":2,\"program\":\"BSCS\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8001/api/students -H "Content-Type: application/json" -d "{\"name\":\"Juan Dela Cruz\",\"email\":\"juan@example.com\",
\"phone\":\"09171234567\",\"year_level\":2,\"program\":\"BSCS\"}"
HTTP/1.1 201 Created
Host: localhost:8001
Date: Tue, 24 Mar 2026 19:14:12 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"id":1,"student_number":"STU-00001","name":"Juan Dela Cruz","email":"juan@example.com","phone":"09171234567","address":null,"date_of_birth":null,"year_level":2,"program":"BSCS","sta
tus":"ACTIVE","created_at":"2026-03-24 19:14:12","updated_at":"2026-03-24 19:14:12"}
```

curl.exe -i http://localhost:8001/api/students/1

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i http://localhost:8001/api/students/1
HTTP/1.1 200 OK
Host: localhost:8001
Date: Tue, 24 Mar 2026 19:18:09 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"id":1,"student_number":"STU-00001","name":"Juan Dela Cruz","email":"juan@example.com","phone":"09171234567","address":null,"date_of_birth":null,"year_level":2,"program":"BSCS","sta
tus":"ACTIVE","created_at":"2026-03-24 19:14:12","updated_at":"2026-03-24 19:14:12"}
```

POST /api/courses followed by GET /api/courses/{id} to create and fetch a course.

curl.exe -i -X POST http://localhost:8002/api/courses ^
  -H "Content-Type: application/json" ^
  -d "{\"name\":\"Introduction to Computing\",\"code\":\"CS101\",\"description\":\"Basic computing concepts\",\"units\":3,\"schedule_day\":\"MWF\",\"schedule_time\":\"9:00-10:00\",\"room\":\"Room 301\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8002/api/courses ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"name\":\"Introduction to Computing\",\"code\":\"CS101\",\"description\":\"Basic computing concepts\",\"units\":3,\"schedule_day\":\"MWF\",\"schedule_time\":\"9:00-10:
0\",\"room\":\"Room 301\"}"
HTTP/1.1 201 Created
Host: localhost:8002
Date: Tue, 24 Mar 2026 19:20:12 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"id":1,"name":"Introduction to Computing","code":"CS101","description":"Basic computing concepts","units":3,"schedule_day":"MWF","schedule_time":"9:00-10:00","room":"Room 301","crea
ted_at":"2026-03-24 19:20:12","updated_at":"2026-03-24 19:20:12","prerequisites":[]}
```

curl.exe -i http://localhost:8002/api/courses/1

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i http://localhost:8002/api/courses/1
HTTP/1.1 200 OK
Host: localhost:8002
Date: Tue, 24 Mar 2026 19:21:24 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"id":1,"name":"Introduction to Computing","code":"CS101","description":"Basic computing concepts","units":3,"schedule_day":"MWF","schedule_time":"9:00-10:00","room":"Room 301","crea
ted_at":"2026-03-24 19:20:12","updated_at":"2026-03-24 19:20:12","prerequisites":[],"required_by":[]}
```

POST /api/enrollments followed by GET /api/enrollments/{id} (and optionally list-by-student) to create and fetch enrollment linking the two.

curl.exe -i -X POST http://localhost:8003/api/enrollments ^
  -H "Content-Type: application/json" ^
  -d "{\"student_id\":1,\"course_id\":1}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?  curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 201 Created
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:22:52 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"id":1,"student_id":1,"course_id":1,"status":"ENROLLED","created_at":"2026-03-24 19:22:52","updated_at":"2026-03-24 19:22:52","student":{"id":1,"student_number":"STU-00001","name":"Juan Dela Cruz","email":"juan@example.com","phone":"09171234567","address"
:null,"date_of_birth":null,"year_level":2,"program":"BSCS","status":"ACTIVE","created_at":"2026-03-24 19:14:12","updated_at":"2026-03-24 19:14:12"},"course":{"id":1,"name":"Introduction to Computing","code":"CS101","description":"Basic computing concepts",
"units":3,"schedule_day":"MWF","schedule_time":"9:00-10:00","room":"Room 301","created_at":"2026-03-24 19:20:12","updated_at":"2026-03-24 19:20:12","prerequisites":[],"required_by":[]}}curl: (6) Could not resolve host: curl.exe
```

curl.exe -i http://localhost:8003/api/enrollments/1

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i http://localhost:8003/api/enrollments/1
HTTP/1.1 200 OK
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:26:06 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"id":1,"student_id":1,"course_id":1,"status":"ENROLLED","created_at":"2026-03-24 19:22:52","updated_at":"2026-03-24 19:22:52","student":{"id":1,"student_number":"STU-00001","name":"
Juan Dela Cruz","email":"juan@example.com","phone":"09171234567","address":null,"date_of_birth":null,"year_level":2,"program":"BSCS","status":"ACTIVE","created_at":"2026-03-24 19:14:
12","updated_at":"2026-03-24 19:14:12"},"course":{"id":1,"name":"Introduction to Computing","code":"CS101","description":"Basic computing concepts","units":3,"schedule_day":"MWF","sc
hedule_time":"9:00-10:00","room":"Room 301","created_at":"2026-03-24 19:20:12","updated_at":"2026-03-24 19:20:12","prerequisites":[],"required_by":[]}}
```

These steps should produce 201 Created for successful creations and 200 OK for successful reads.

Validation and client error scenarios (400)

To test validation, I intentionally send bad JSON:

For the Student Service: missing email, or an invalid email format.

Student – missing email

curl.exe -i -X POST http://localhost:8001/api/students ^
-H "Content-Type: application/json" ^
-d "{\"name\":\"No Email Student\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8001/api/students ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"name\":\"No Email Student\"}"
HTTP/1.1 400 Bad Request
Host: localhost:8001
Date: Tue, 24 Mar 2026 19:28:18 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"VALIDATION_ERROR","message":"Name and email are required"}
```

Student – invalid email

curl.exe -i -X POST http://localhost:8001/api/students ^

-H "Content-Type: application/json" ^

-d "{\"name\":\"Bad Email\",\"email\":\"not-an-email\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8001/api/students ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"name\":\"Bad Email\",\"email\":\"not-an-email\"}"
HTTP/1.1 400 Bad Request
Host: localhost:8001
Date: Tue, 24 Mar 2026 19:29:20 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"VALIDATION_ERROR","message":"Invalid email format"}
```

For the Course Service: missing name or code.

curl.exe -i -X POST http://localhost:8002/api/courses ^

-H "Content-Type: application/json" ^

-d "{}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8002/api/courses ^
More?   -H "Content-Type: application/json" ^
More?   -d "{}"
HTTP/1.1 400 Bad Request
Host: localhost:8002
Date: Tue, 24 Mar 2026 19:30:00 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"VALIDATION_ERROR","message":"Name and code are required"}
```

For the Enrollment Service: missing course_id or using non-numeric IDs.

Course – missing name/code

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?   -d "{}"
HTTP/1.1 400 Bad Request
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:31:23 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"VALIDATION_ERROR","message":"student_id and course_id are required"}
```

Enrollment – non-numeric IDs

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{\"student_id\":\"abc\",\"course_id\":\"xyz\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"student_id\":\"abc\",\"course_id\":\"xyz\"}"
HTTP/1.1 400 Bad Request
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:34:00 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"VALIDATION_ERROR","message":"student_id and course_id must be numeric"}
```

Each case should return 400 Bad Request with a JSON body that includes error: "VALIDATION_ERROR" and a descriptive message.

Duplicate and conflict scenarios (409)

I verify that uniqueness constraints are enforced at the application level:

Submitting the same email twice to the Student Service.

curl.exe -i -X POST http://localhost:8001/api/students ^

-H "Content-Type: application/json" ^

-d "{\"name\":\"Duplicate Student\",\"email\":\"juan@example.com\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8001/api/students ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"name\":\"Duplicate Student\",\"email\":\"juan@example.com\"}"
HTTP/1.1 409 Conflict
Host: localhost:8001
Date: Tue, 24 Mar 2026 19:35:15 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"DUPLICATE_EMAIL","message":"A student with this email already exists"}
```

Submitting the same code twice to the Course Service.

curl.exe -i -X POST http://localhost:8002/api/courses ^

-H "Content-Type: application/json" ^

-d "{\"name\":\"Duplicate Course\",\"code\":\"CS101\"}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8002/api/courses ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"name\":\"Duplicate Course\",\"code\":\"CS101\"}"
HTTP/1.1 409 Conflict
Host: localhost:8002
Date: Tue, 24 Mar 2026 19:36:04 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"DUPLICATE_CODE","message":"A course with this code already exists"}
```

Creating the same (student_id, course_id) enrollment pair twice in the Enrollment Service.

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{\"student_id\":1,\"course_id\":1}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 409 Conflict
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:37:54 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"DUPLICATE_ENROLLMENT","message":"Student is already enrolled in this course"}
```

These cases should return 409 Conflict with specific error codes such as DUPLICATE_EMAIL, DUPLICATE_CODE, or DUPLICATE_ENROLLMENT.

Not-found scenarios (404)

I send requests with obviously nonexistent IDs:

GET /api/students/99999

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i http://localhost:8001/api/students/99999
HTTP/1.1 404 Not Found
Host: localhost:8001
Date: Tue, 24 Mar 2026 19:38:34 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"STUDENT_NOT_FOUND","message":"Student with ID 99999 not found"}
```

GET /api/courses/99999

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i http://localhost:8002/api/courses/99999
HTTP/1.1 404 Not Found
Host: localhost:8002
Date: Tue, 24 Mar 2026 19:38:54 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"COURSE_NOT_FOUND","message":"Course with ID 99999 not found"}
```

GET /api/enrollments/99999

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i http://localhost:8003/api/enrollments/99999
HTTP/1.1 404 Not Found
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:39:16 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"ENROLLMENT_NOT_FOUND","message":"Enrollment with ID 99999 not found"}
```

and also test cross-service cases in the Enrollment Service by trying to enroll a nonexistent student or course. Direct not-found scenarios are expected to return 404 with codes such as STUDENT_NOT_FOUND, COURSE_NOT_FOUND, or ENROLLMENT_NOT_FOUND, while cross-service not-found scenarios should result in STUDENT_NOT_FOUND or COURSE_NOT_FOUND from the Enrollment Service after it checks the other services.

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{\"student_id\":99999,\"course_id\":1}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"student_id\":99999,\"course_id\":1}"
HTTP/1.1 404 Not Found
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:40:05 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"STUDENT_NOT_FOUND","message":"Student with ID 99999 not found"}
```

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{\"student_id\":1,\"course_id\":99999}"

```
C:\laragon\www\ITPFL\lab-2>curl.exe -i -X POST http://localhost:8003/api/enrollments ^
More?   -H "Content-Type: application/json" ^
More?   -d "{\"student_id\":1,\"course_id\":99999}"
HTTP/1.1 404 Not Found
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:41:00 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"COURSE_NOT_FOUND","message":"Course with ID 99999 not found"}
```

Dependency failure scenarios (503 and 504)

Finally, I simulate two types of dependency failures from the point of view of the Enrollment Service:

Service down (503): I stop the Student Service and then attempt to create a new enrollment using a valid student_id and course_id. The Enrollment Service cannot reach the Student Service and should respond with 503 Service Unavailable and an error like SERVICE_UNAVAILABLE.

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{\"student_id\":1,\"course_id\":1}"

```
λ curl.exe -i -X POST http://localhost:8003/api/enrollments ^   -H "Content-Type: application/json" ^   -d "{\"student_id\":1,\"course_id\":1}"
HTTP/1.1 503 Service Unavailable
Host: localhost:8003
Date: Tue, 24 Mar 2026 19:49:09 GMT
Connection: close
X-Powered-By: PHP/8.4.19
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Accept

{"error":"SERVICE_UNAVAILABLE","message":"Student Service is currently unavailable"}
```

Service too slow (504): I temporarily insert an artificial delay (such as a sleep(10) call) into the Student Service's handler so that it exceeds the Enrollment Service's HTTP client timeout. When I then attempt to create an enrollment, the client times out and the Enrollment Service returns 504 Gateway Timeout with an error like GATEWAY_TIMEOUT.

curl.exe -i -X POST http://localhost:8003/api/enrollments ^

-H "Content-Type: application/json" ^

-d "{\"student_id\":1,\"course_id\":1}"

D. Use of curl -i

Every request in my test plan is executed using curl -i instead of plain curl. The -i flag prints the HTTP response headers along with the body, so I can see both the exact status code (for example, HTTP/1.1 409 Conflict) and the JSON error payload. This is important because the purpose of the lab is to verify status codes and error contracts, not only to look at the body contents.

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

The test scenarios for the TechVault product-ordering system produced the expected HTTP status codes and JSON responses. The normal-flow tests showed that the API can reliably list all products, retrieve a specific product by ID, accept a valid order, and immediately reflect stock reductions in subsequent GET requests. Cart-based operations also behaved correctly: items could be added with different quantities, viewed through the cart API, updated, removed, and finally checked out to create an order while clearing the cart and updating inventory.

The remaining scenarios focused on invalid input and error handling. Requests using non-existent product IDs returned "Product not found" with the appropriate 4xx status. Zero, negative, and overly large quantities were rejected as invalid or exceeding stock, and missing required fields such as productId or quantity produced validation errors rather than silent failures. An intentionally incorrect route returned a JSON 404 response instead of a generic HTML error page. Together, these results indicate that the application enforces its business rules consistently and fails safely when clients send incorrect or incomplete data.

IV. LAB DISCUSSIONS / COMPUTATIONS

Health and normal operations

All three services responded to their /api/health endpoints with 200 OK, confirming that they were running and reachable. The basic CRUD operations (creating and reading a student, course, and enrollment) also worked as expected. Student and Course services returned 201 for creations and 200 for reads, while the Enrollment Service successfully linked existing student and course records and returned enriched enrollment data.

Validation behaviour (400)

When I sent requests with missing required fields or invalid formats, each service responded with 400 Bad Request and a JSON body that included error: "VALIDATION_ERROR". For the Student Service, missing email and invalid email formats were caught correctly. For the Course Service, missing name or code triggered validation errors. For the Enrollment Service, missing course_id or using non-numeric IDs resulted in clear validation messages. These responses make it straightforward for clients to understand and correct their input.

Duplicate handling (409)

Submitting a second student with the same email consistently produced a 409 Conflict with a duplicate-email error code. Reusing the same course code produced a 409 Conflict with a duplicate-code error. Repeating an enrollment request with the same student_id and course_id also produced a 409 Conflict with a duplicate-enrollment error. In all of these cases, the services clearly differentiated between syntactically valid requests and requests that violate data constraints.

Not-found scenarios (404)

Requests for students, courses, or enrollments with IDs that do not exist correctly returned 404 Not Found. The JSON bodies included specific error codes such as STUDENT_NOT_FOUND, COURSE_NOT_FOUND, or ENROLLMENT_NOT_FOUND, often alongside messages that included the invalid ID. In cross-service tests, the Enrollment Service did not create new enrollments when either the student or the course was missing; instead, it mapped the upstream not-found conditions to STUDENT_NOT_FOUND or COURSE_NOT_FOUND and returned 404 to the client.

Dependency failures (503 and 504)

When I stopped the Student Service and attempted to create a new enrollment, the Enrollment Service detected that its dependency was unreachable and returned 503 Service Unavailable with an appropriate error code and message. When I slowed the Student Service with an artificial delay that exceeded the configured timeout, the Enrollment Service responded with 504 Gateway Timeout. These tests showed that the Enrollment Service can distinguish between “service down” and “service too slow” and report each situation accurately using different 5xx codes.

Consistency of error format

Across all services and test cases, error responses followed a consistent JSON structure with error and message fields. This uniformity held for validation errors, not-found errors, conflicts, and dependency failures. The consistency of the error format is as important as the correctness of the status codes, because clients can rely on a single error-handling strategy.

V. CONCLUSIONS

The lab successfully achieved its objective of implementing and validating a Product Ordering API that enforces realistic business rules over product inventory and customer orders. By structuring the system into presentation, business logic, and data layers, the implementation maintained a clear separation of concerns, making it easier to reason about how requests are processed and how rules are enforced. Centralized validation logic ensured that constraints on product existence, quantity, and stock were consistently applied across all ordering paths, whether via direct orders or cart checkouts. Systematic testing using curl demonstrated that the endpoints produced correct JSON outputs for both normal and erroneous cases, and that stock levels and cart contents stayed synchronized with user actions. Overall, the exercise strengthened understanding of how to design and implement backend services that not only expose endpoints but also rigorously enforce domain-specific rules, providing a solid foundation for more complex transactional systems.

References

Fielding, R. T., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content. IETF RFC 7231.

Mozilla Developer Network. “HTTP response status codes.” https://developer.mozilla.org/

Laravel Documentation. “HTTP Client” and “Validation.” https://laravel.com/docs/

Course handouts and laboratory instructions for Microservices and System Integration (Semester materials provided by the instructor)